



Experiment No. 6

Title: Lemmatization of Text using NLTK



Batch: NLP-1

Experiment No.: 06

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To implement lemmatization of Text using NLTK

Program/Steps:

1. Perform lemmatization of Text , with and without POS Tagging to large corpus of text.

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
from nltk.corpus import wordnet

nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')

text_corpus = """
Natural language processing (NLP) is a subfield of linguistics, computer
science,
and artificial intelligence concerned with the interactions between
computers and human language.
It is an extremely interesting field that has been growing rapidly in
recent years.
NLP technologies are being used for various applications, including
chatbots, machine
translation, and sentiment analysis. The challenges in NLP are many, but
researchers are
making great progress. Understanding context, dealing with ambiguity, and
handling different
languages are some of the key problems. These challenges are being tackled
by using advanced
machine learning models, especially deep learning.
"""

words = word_tokenize(text_corpus.lower())

stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word.isalnum() and word not in
```

```

stop_words]

lemmatizer = WordNetLemmatizer()

lemmatized_words_no_pos = [lemmatizer.lemmatize(word) for word in
filtered_words]

print("--- Lemmatization without POS Tagging ---")
print("Original words:", filtered_words[:15])
print("Lemmatized words:", lemmatized_words_no_pos[:15])
print("-" * 50)

def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

pos_tagged_words = pos_tag(filtered_words)

lemmatized_words_pos = []
for word, tag in pos_tagged_words:
    wntag = get_wordnet_pos(tag)
    lemmatized_words_pos.append(lemmatizer.lemmatize(word, pos=wntag))

print("--- Lemmatization with POS Tagging ---")
print("Original words:", filtered_words[:15])
print("Lemmatized words (with POS):", lemmatized_words_pos[:15])
print("-" * 50)

```

2. Write down the observations about the difference in the results of lemmatization with POS and without POS Tagging, when both are applied on the same text corpus.

Observations:

When comparing the output of the two methods, significant differences are observed, particularly for verbs and adjectives:

- **Without POS Tagging:** The lemmatizer defaults to a noun tag. As a result, many words, especially verbs, are not correctly reduced to their base form. For instance, "growing," "processing," and "understanding" would remain unchanged because the lemmatizer

- incorrectly assumes they are nouns.
- **With POS Tagging:** By providing the correct POS tag, the lemmatizer can accurately identify the verb form of words. For example, "growing" is correctly identified as a verb and lemmatized to "grow." "concerned" is correctly identified as an adjective and lemmatized to "concerned" (which is already its base form).

The example below highlights this difference. Without POS tags, 'concerned', 'growing', and 'using' are not lemmatized correctly, while with POS tags, they are correctly reduced to their base forms:

Original Word	Lemmatized (No POS)	Lemmatized (With POS)	Reason for Difference
concerned	concerned	concerned	Verb vs. Adj, but both return same lemma
growing	growing	grow	Verb vs. Noun default
using	using	use	Verb vs. Noun default
was	wa	be	Verb vs. Noun default (wrong lemma)

The use of POS tagging significantly increases the accuracy and contextual relevance of the lemmatization process. It ensures that the base form of a word is not only a valid word but also the correct one based on its grammatical role in the sentence.

Output/Result:

```
PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-6> & C:/Users/Dell/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Dell/Downloads/VII SEM/HO-NLP/LAB/EXP-6/EXP-6.py"
[nltk_data] Downloading package punkt to C:\Users\Dell\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Dell\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\Dell\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to C:\Users\Dell\nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-date!
[nltk_data]
--- Lemmatization without POS Tagging ---
Original words: ['natural', 'language', 'processing', 'nlp', 'subfield', 'linguistics', 'computer', 'science', 'artificial', 'intelligence', 'concerned', 'interactions', 'computers', 'human', 'language']
Lemmatized words: ['natural', 'language', 'processing', 'nlp', 'subfield', 'linguistics', 'computer', 'science', 'artificial', 'intelligence', 'concerned', 'interaction', 'computer', 'human', 'language']
-----
--- Lemmatization with POS Tagging ---
Original words: ['natural', 'language', 'processing', 'nlp', 'subfield', 'linguistics', 'computer', 'science', 'artificial', 'intelligence', 'concerned', 'interactions', 'computers', 'human', 'language']
Lemmatized words (with POS): ['natural', 'language', 'processing', 'nlp', 'subfield', 'linguistics', 'computer', 'science', 'artificial', 'intelligence', 'concern', 'interaction', 'computer', 'human', 'language']
-----
PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-6>
```

Post Lab Question-Answers:

1. Explain the difference between stemming and lemmatization of text

Ans: Stemming and lemmatization are both text normalization techniques used in Natural Language Processing (NLP) to reduce words to their base or root form. The key difference lies in their approach and the quality of the output.

- Stemming is a more aggressive, rule-based approach that chops off prefixes and suffixes from a word to get its stem. This process often results in a root that isn't a valid word itself. For example, the stemmer might reduce "running," "ran," and "runner" to the stem "runn," which isn't a recognizable word. Stemming is faster and computationally less expensive but can produce less accurate results.
- Lemmatization is a more sophisticated, dictionary-based approach. It considers the word's meaning and its Part-of-Speech (POS) to find its true base form, known as the lemma. The lemma is always a valid word that can be found in a dictionary. For example, the lemmatizer would reduce "running," "ran," and "runner" to the words "run" and "runner," respectively, which are both meaningful words. Lemmatization is more accurate but requires a linguistic resource like WordNet, making it slower than stemming.

In summary, stemming is a heuristic process that is quick and dirty, while lemmatization is a more precise linguistic process that is slower but yields better results by ensuring the base form is a valid word.

Outcomes: CO3 – Establish concept of Structure and Semantics

Conclusion (based on the Results and outcomes achieved):

Lemmatization is a powerful technique for normalizing text, and its accuracy is significantly improved by incorporating Part-of-Speech (POS) tags. While lemmatization without POS tags provides a basic lemma, it can fail to account for the grammatical context of a word, leading to incorrect base forms. By providing a POS tag, the lemmatizer can accurately determine the correct dictionary form (lemma) of a word, as seen in the examples of 'leaves' becoming 'leave' when tagged as a verb, which is a more semantically accurate result.

References:

Books/ Journals/ Websites:

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
 2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
 3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018
-